IMPLEMENTATION PROPOSAL / REVISION 6

Discrete-event service simulator

Sustained state, cancellation races, recovery contracts and measurable browser pressure

DECISION

Ship the pure deterministic engine and separately coded JavaScript oracle first; then build a fixed-topology Scene plus Petri-style owned SVG overlay, explicit connected-host and gallery lifecycle guards, and a fault-swept controller. Extend the implemented harness with **W6** for clean runs and a separate **W5** simulator executor for injected recovery. A swept **Scene::set_data_values** path is required before final acceptance.

Design axis	Binding decision
Algorithm	Integer-tick, stable-ordered priority queue; explicit retries, deadlines, cancellations and server outages.
UI	2–4 servers, fixed Scene topology and Petri-style SVG overlay; a live data node is mandatory for the final sustained-crate test.
Playback	One pending timer and one pending animation frame at most; invalidation tokens on all stops and history changes.
Recovery	Explicit host-is-connected guard; one immediate bounded repair; documented failure sweeps before set_* use.
Proof	Separately coded JS model; verified E0 19/12/13; RetryWait cancellation/timeout fixtures; defined W6 batches and W5 target.

Prepared 6 November 2026 | Repository: **ChrisWhealy/svg-dom-graph**, main 65511d1c64c3, crate 0.2.48. | Status: **S0 approved; S1 source starter tested in JavaScript only; Rust/WASI and repository integration outstanding**.

Prior evidence: 4 November coverage audit, performance-harness design rev 6 and implemented harness Phase 5, and repository Petri-net controller/scheduler/trace implementations. The harness currently defines W1–W5. W6 and its manifest schema changes are proposals, not existing APIs.

Source status: the S1 starter is distributed in a local ZIP only, not installed as service-sim-engine/ in the repository. The package is review material, not a committed stage or verified implementation. Integrate in a reviewable feature branch only after compile/test gates; none of the Stage S1 merge gates are claimed.

Approval policy: S0 is approved; S1 (engine + JavaScript oracle) has started and remains independent of UI/harness uncertainties. S0 must settle the mandatory Scene setter, content-group compatibility, synchronous gallery guard, empirically qualified W6 activity bounds, and the harness-owned Layer C Play mark and cancellation protocol of sections 8.3–8.4. E0 hashes are frozen only after independent implementation agreement. Timings are never correctness gates.

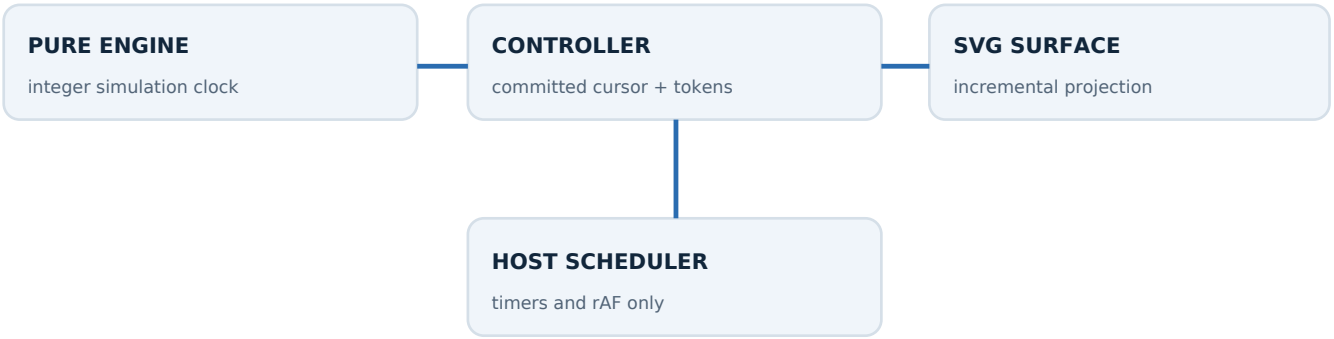
1 Goals, scope and non-goals

The demo is a sustained, stateful client/service simulation, not a collection of isolated graph transitions. It must create a continuous stream of logically meaningful changes while users pause, seek, switch configurations, interrupt playback, and recover from deliberate faults.

Required coverage	Evidence of success
Sustained state	Multiple servers remain occupied across many simulated timestamps; a bounded waiting queue, deadlines, retries and terminal outcomes all evolve without scene reconstruction.
Cancellation	Late host timer/frame callbacks never change the logical cursor, DOM, focus or notices after a stop, seek, scenario switch or teardown.
Failure recovery	Injected renderer failures are repaired from the committed state; scheduling failures stop cleanly; simulated outages follow specified business rules.
Traceability	Same seeded run yields identical event stream, timestamp slices, state and checkpoint hashes on native Rust and WASM.
Harness	A/B/C metrics with correct timing boundaries; deterministic lifecycle and resource-count gates; W5 simulator-target recovery, separately reported from clean W6 runs.

Out of scope: realistic queueing-theory calibration, distributed concurrency, persistent browser storage, network services, workers, a new public `svg-dom-graph` API, and per-event graphical animation for a 100k-event run. Little's law or analytic distributions may be diagnostics, never a substitute for oracle agreement.

2 Four boundaries that must not be blurred



The engine owns an integer virtual clock and a deterministic future-event heap. The host scheduler only chooses when the next already determined timestamp slice is presented. No simulated arrival, deadline or service completion is implemented with a browser timer.

The controller owns a committed batch cursor, a generation for the selected history, a playback epoch, dirty-surface state and pending host callback handles. The SVG surface is a projection, never the source of truth. The harness observes pure engine work (A), synchronous DOM mutation (B) and input-to-presentation behaviour (C), without merging their figures.

3 Deterministic model

3.1 Configuration and identities

Use checked unsigned integer simulated ticks, never wall-clock elapsed time. SplitMix64 with fixed stream splitting determines arrivals, priorities, durations and scripted cancellations. All random draws and script encodings are canonical; no JavaScript Math.random, browser timing or host callbacks change the model. A scenario configuration is immutable once the run begins.

Field	Proposed binding rule
servers	Default 3; supported 2–4; stable S0..S3 IDs.
queue_capacity	Default 12; allowed 1–64; waiting only, excluding running and retry-wait jobs.
service_ticks	Positive per-attempt integer 1..=12; fixture-specific overrides are explicit.
max_attempts	Default 2; 1–3 allowed. Backoff after failed attempt n = 2 * 2^(n-1) ticks.
retry_deferrals	**Cap 4** full-queue deferrals per retry episode. On the fifth full-queue eligibility pop, mark Failed; no extra retry is scheduled.
deadline	Absolute tick strictly later than admission tick; applies to running and waiting.
arrival identity	Monotonic u64 numeric ID allocated only when an Arrival event pops; never reused. External stable job keys may be resolved before that pop.
resource ceilings	Validate maximum arrivals, future heap, pops, slices, horizon and generated input bytes before allocation.

Same-tick source mixing: fixed scripts are preloaded in their authored order before running. The seed-driven source schedules exactly one next arrival when the preceding generated arrival pops, with a positive gap of at least one tick. If a generated arrival coincides with a scripted arrival, the script already has the lower insertion sequence and therefore precedes it at rank 6. The independent input hash is computed from the full canonical recipe (script plus deterministic generated stream), not from the processing order; the actual pop hash separately checks scheduling. Tests must include mixed-source collisions.

Identity binding: authored inputs use stable `job_key` references, not preassigned numeric IDs. On each rank-6 Arrival pop, allocate the next u64 numeric job ID (including arrivals that are immediately rejected), then bind its source key to that ID. An earlier rank-2 cancel for the same key sees no admitted job and remains a no-op. A same-tick scripted arrival gets the earlier ID when inserted ahead of the generated arrival. E0 labels J0–J5 denote that resulting arrival-pop order. Freeze the key→ID mapping in the input and pop traces.

3.2 State and invariant set

`JobState` is exactly one of `NotArrived, Queued, Running(server,attempt), RetryWait(eligible_tick), Completed, TimedOut, Cancelled, Rejected, Failed(reason)`. A `Failed(reason)` carries `attempts_exhausted` or `retry_deferrals_exhausted`, encoded with stable discriminants. A server is `UpIdle, UpBusy(job,attempt,due), Down`. A running job has exactly one corresponding busy server, and vice versa.

- **Conservation:** attempted arrivals = admitted + rejected; admitted = queued + running + retry_wait + completed + timed_out + cancelled + failed. Count each job once in exactly one class.
- **Capacity:** waiting length ≤ queue_capacity; occupied servers ≤ configured server count; no two servers hold the same job.
- **Time:** popped event time never moves backward; all newly scheduled events have strictly greater tick than the current timestamp; duration and retry delays are positive.
- **Work:** an attempt may complete at most once; stale/cancelled completion and deadline events may pop but have no effect; a deadline at exactly completion time is won by completion.
- **History:** all counters use checked integers; every job transition is legal; resource counts and terminal counters match the independent reference after every timestamp batch.

3.3 Heap ordering and one atomic timestamp slice

Store each event as `(tick, kind_rank, insertion_sequence, payload)` in a min-heap. The insertion sequence is allocated monotonically when a future event is created. Pop every event at the minimum timestamp, including newly discovered stale events; process the following ranks in order, then run deterministic dispatch once. A snapshot is committed only after the complete timestamp slice.

Rank	Event	Tie-breaking / consequence
0	ServiceComplete	Completes before deadline, explicit cancel or outage on the same tick. Ignore a stale attempt token.
1	ServerFail	If UpBusy: abort attempt, invalidate completion; if UpIdle: make Down; if already Down: no-op.
2	ExternalCancel	Terminate a non-terminal admitted job, even if running. Missing/terminal/rejected targets are no-ops.
3	Deadline	Expire a still-active job, including one running. Exactly-on-deadline completion already won at rank 0.
4	ServerRecover	If Down: mark UpIdle; if already Up: no-op. Only final dispatch assigns work.
5	RetryEligible	Re-queue if capacity permits; full queue defers by one tick, at most four times.
6	Arrival	Admit to waiting queue if space exists; otherwise terminal Rejected. No direct bypass of a full queue.
post	Dispatch	Order jobs by priority (high first), then queue-entry tick, then job ID. Order idle servers by ID.

Capacity is evaluated before final dispatch. Same-tick arrivals can therefore be rejected even while a server is idle if waiting is full. A cancellation scripted for the job's own arrival tick has rank 2, before Arrival at rank 6, and is a logged **no-op** rather than a delayed cancellation. An invalid duplicate failure/recovery is also a logged no-op. At one tick, ServerFail (rank 1) runs before ServerRecover (rank 4): a busy server aborts its attempt, becomes Down, then returns UpIdle for dispatch at the end of the slice. These ordering choices are binding for all fixtures and hashes.

3.4 Failure and retry semantics

Scripted server outages are simulation events, not harness fault injection. On a fail for UpBusy, invalidate the completion token and increment the attempt-failure count; enter RetryWait if attempts remain, or terminal Failed otherwise. A fail for UpIdle makes Down; a fail for Down is a no-op. Recovery from Down produces UpIdle; recovery of an already-up server is a no-op. All outage events, including no-ops, count as heap pops and are included in the event-pop hash.

External cancellation or deadline expiration terminates a Queued, Running or RetryWait job, releases a busy server, and invalidates its attempt token. Dispatch happens once at the end of the slice. RetryEligible encountering a full waiting queue reschedules at tick + 1 for the first four deferrals; the fifth such pop marks the job Failed. A successful retry admission resets the deferral count. A final failed service attempt becomes Failed(attempts_exhausted) immediately without scheduling retry eligibility. A fifth full-queue retry eligibility pop becomes Failed(retry_deferrals_exhausted). All additions are checked for overflow and time monotonicity.

Retain a distinct attempt generation/token. Stale completion and deadline pops remain in the heap and contribute to workload counters, but never mutate terminal jobs. The engine does not remove them from the heap at cancellation time.

4 Hand-audited known-answer fixture: E0

E0 is specified without randomness so the first implementation can be checked against a handwritten ledger before any seeded tests. It has **two servers, queue capacity three, maximum two service attempts**. Explicit service times override the generator.

Tick	Input / event	Expected consequence after the full slice
0	J0 high/4/deadline10; J1 low/6/deadline20	J0 starts S0; J1 starts S1.
1	J2 high/2/deadline8	J2 queues.
2	Fail S0 until tick7; J3 low/2/deadline5	J0 attempt1 aborts, retry eligible at tick4; J3 queues.
4	Stale J0 completion; J0 retry eligible; J4 high/1/deadline9	J0 re-queues; J4 rejected because all three waiting slots are occupied.

Tick	Input / event	Expected consequence after the full slice
5	J3 deadline; J5 low/3/deadline15	J3 times out; J5 admitted into the freed waiting slot.
6	J1 completion; cancel J5	J1 completes; J5 cancelled; J2 begins S1, due tick8.
7	Recover S0	J0 attempt2 begins S0; overridden duration 2, due tick9.
8	J2 completion and deadline	Completion wins the tie; deadline pop is stale.
9	J0 completion	J0 completes.
10,15,20	Remaining deadline events	All are stale and do not change terminal states.

FIXED ACCEPTANCE TUPLE

19 heap pops, 12 nonempty timestamp slices, 13 committed snapshots including index 0. Arrival attempts = 6; admitted = 5; rejected = 1. Terminal jobs: Completed {J0,J1,J2}, TimedOut {J3}, Cancelled {J5}, Rejected {J4}; zero Failed. One aborted attempt, one retry eligibility, one outage and one recovery. An independent reviewer reproduced the counts; exact canonical hashes are still to be frozen from two implementations.

There are **five stale pops**: J0 attempt-one completion, J2 deadline, and the remaining deadlines at ticks 10, 15, 20. Stale-only ticks remain separately addressable slices. E0 does not test timeout/cancellation of a running job, nor the terminal Failed state: those belong to E1–E4 below. No service event is scheduled for rejected J4.

4.1 Required companion fixtures

Fixture	Deliberate stress	Oracle/gate
E1 — running + cancel	Cancel a Running job before completion; timeout another Running job. Separate RetryWait cancellation probe E1-RC.	No duplicate terminal accounting; each superseded completion/eligibility pop is stale.
E2 — failure + timeout	Max-attempt failure; four full-queue retry deferrals, then Failed on fifth; RetryWait deadline probe E2-RT.	Reason codes distinct; pending RetryEligible pop is stale after timeout.
E3 — source/tie	Duplicate fail/recover; fail+recover same timestamp; self-cancel at arrival tick; collide seeded and script arrivals.	No-op ledger and insertion-order hash fixed.
E4 — host/control	Hide/show during Play; inject host detach between synchronous DOM writes; fail mandatory data setter and repair.	Post-draw connectivity guard catches detach; committed cursor unchanged.

4.2 Binding RetryWait regression probes

E1-RC (cancel while RetryWait): one server, queue capacity two, max two attempts. J0 arrives tick 0, starts duration 5, deadline tick 20. ServerFail(S0) at tick 1 aborts attempt 1 and schedules RetryEligible for tick 3 (2-tick backoff). Cancel(J0) at tick 2 changes RetryWait to Cancelled. Its RetryEligible pop at tick 3, aborted completion at tick 5 and deadline at tick 20 are all stale. Expected: **6 pops, 6 nonempty slices, 7 snapshots**; terminal Cancelled, no attempt 2 and no busy server. The server remains Down, which is legal.

E2-RT (deadline while RetryWait): identical configuration except J0 deadline tick 2 and no external cancel. ServerFail at tick 1 schedules RetryEligible tick 3; deadline at tick 2 changes RetryWait to TimedOut. RetryEligible tick 3 and aborted completion tick 5 are stale. Expected: **5 pops, 5 slices, 6 snapshots**; terminal TimedOut, no attempt 2. These tiny fixtures run before seeded differential tests and pin eligibility invalidation. Their expected counts are design derivations to confirm independently, unlike the already checked E0.

5 Architecture, retention and projection

5.1 Modules and ownership

Module	Responsibility / ownership
`engine/`	Pure engine, event heap, job and server tables, event generator, legal transition checks.
`reference-js/`	Independent JavaScript reference with sorted-array events, separate transition functions and no shared Rust engine calls.
`history/`	Immutable inputs, timestamp index and optional measured checkpoints, exact canonical replay hashes.
`projection/`	Pure function from committed state and view to fixed slots, labels, accessible descriptions and highlights.
`controller/`	Logical cursor, history generation, playback epoch, dirty repair, scheduled seek coalescing and recovery.
`surface/`	Persistent svg-dom-graph Scene, delta/full writes, accessibility controls and focus preservation.
`scheduler/`	Browser adapter and deterministic test clock; both implement a small timer/rAF/cancel trait.
`harness/`	W6 clean performance/correctness/endurance; W5 simulator target for fault and recovery runs.

Reuse rule: the Petri controller and scheduler types are currently module-private (`pub(crate)` in a demo module). Reuse their tested contracts first. Extract a shared private controller/scheduler primitive only if S0 demonstrates equivalence without creating a shared mega-controller or changing the library's published API.

S1 crate location and independence: create unpublished workspace member `service-sim-engine/`, with `publish = false`, no `web-sys` and no dependency from the published library. Compile its pure library natively and under `wasm32-wasip1` with the existing 32-bit runner; add native tests, clippy and WASI checks to repository CI. The JS sorted-array oracle is authored from this specification, not translated from the Rust heap engine. When they disagree, adjudicate by the binding text and record the first divergent pop/slice, cause and regression in a discrepancy ledger before changing either implementation. E1-RC and E2-RT expected counts are design derivations until both independently agree.

5.2 Buildable fixed-slot projection: exact API map

The baseline is the Petri overlay pattern in `demo-app/src/petri_net/scene.rs`, not a hypothetical Scene node-visibility mechanism. Build a fixed Scene once using `SvgRoot::attach`, `Scene::new`, `add_node_with` (server frames) and `add_edge_with` (stable routing). The Petri demo locates the `.svg-dom-graph-content` group and creates a fixed application-owned SVG overlay within the same coordinate system. Copy this known construction style, including stable logical slot keys, and validate the group contract in simulator stage S0.

Visual role	Exact write surface	Fallback / prohibited assumption
Servers + fixed topology	Scene::new, add_node_with, add_edge_with at initial construction.	No dynamic add/remove in steady state; no runtime node relabelling.
Queue window: 12 slots	Pre-created SVG overlay text/rect/group elements; `web_sys::Element` attribute/text writes via Petri dom helpers.	Unused = empty text, inactive class and explicit aria-hidden; not Scene::hide_node.
Counters/status + selected job	Fixed overlay text elements with `set_text_content` and checked attribute helpers.	Selection is separate from focus; no DOM readback drives the model.
Required numeric state panel	Fixed data node created with `add_data_node`; update via `Scene::set_data_values` after sweep.	Mandatory for S5; one successful setter call on each visible committed slice, even stale-only slices. This is required for **every displayed S2 slice**, not only the first 1,000; the latter is a minimum acceptance floor.

Visual role	Exact write surface	Fallback / prohibited assumption
Optional time-series panel	`Scene::set_plot_items` **only after sweep**.	No guessed idempotence, rollback or redraw guarantee.
Edge labels, input values	`set_edge_label`, `set_text_input_value` are audit targets, **not** projection dependencies.	Do not add them to runtime until their sweep contracts are documented.

This no-new-public-API proposal depends only on existing fixed Scene topology and application-owned DOM writes. Scene internals are not mutated directly; the overlay is the demo-owned subtree. Any inaccessible coordinate/transform or focus requirement is a stage S0 **no-go** until a fixed-slot proof is shown. Never claim that the library has remove_node, dynamic ordinary-node text or show/hide functions.

A 12-slot waiting window shows the ordered head, with "+N more" and a selected-job detail outside it. Queue slots remain allocated even when blank. Stable server/queue ids and logical DOM keys do not change on ordinary Next, Seek or playback. A pure projection yields the same slot texts, attributes, accessibility names and ordering for delta and full redraw.

Proof boundary: the owned overlay proves application/controller/DOM lifecycle behaviour, not sustained crate mutation. The S5 claim about `svg-dom-graph` requires a live Scene data node that is updated by `set_data_values` on every displayed slice, without rebuilding Scene. Its data include the slice ordinal (so each write changes content) and a stable selection of counters. Run at least **1,000 consecutive displayed-slice commits** through that same node, with a fresh whole-scene checkpoint after each batch and bounded element/listener counts. A failed setter sweep can block S4/S5: overlay-only is a permitted development fallback but **not a successful sustained-library acceptance**. Replacing the setter requires a documented spec revision, not a silent downgrade.

DOM integration contract: `.svg-dom-graph-content` is presently an internal class also used by existing demos. S0 must either document it as a supported transformed content-group hook and pin it with a library-level Chrome integration test, or supply an equally stable supported hook. Test existence, uniqueness, common view transform for overlay and fixed nodes, and behaviour after zoom/pan and scene rebuild. This is a semver-relevant DOM compatibility commitment, not a new Rust public method. If the class becomes supported, record the S0 choice and a changelog.md entry so a subsequent refactor cannot silently break consumers.

5.3 History and random access

Snapshot 0 is the initial state; snapshot k follows complete simulated timestamp slice k. Full replay, repeated Next and Seek must produce the same canonical model checkpoint and the same visible projection. Headless 100k-pop runs stream hashes rather than retaining 100k SVG frames. Any replay checkpoint must contain the entire continuation state: jobs, servers, waiting order, retry heap, event sequence, generator cursor, counters and virtual tick.

Defer unnecessary machinery. First measure seeking at 1k, 10k and 100k pops with direct deterministic replay from the start and with optional 128-slice checkpoints. Record p50/p95 and worst work, allocation and responsiveness. Add a bounded checkpoint cache only if measured need justifies it; do **not** hard-code a 32 MiB cache or mandatory 64 images. Likewise, cooperative chunking is optional for normal use, but required as a forced test mode with a one-slice budget and generation-token invalidation to exercise cancellation races. Keep expensive staging isolated from committed cursor and Scene; a cancelled stage cannot mutate the displayed model.

A failed projection never commits a new cursor. Full redraw rewrites the existing scene and overlay rather than reconstructing Scene (which would reset pan/zoom, focus and ids). A legitimate scenario change may rebuild, but must save and restore the current view transform and an equivalent focus target under a documented policy.

6 Controller, lifecycle and cancellation

6.1 Commit ordering, host guard and tokens

```
Controller { committed_slice, committed_model, scenario_generation,
             playback_epoch, seek_token, timer_handle, frame_handle,
             dirty, visible, host_root, disposed, selected_job, view }

render_then_commit(target):
   if !live_and_visible(): stop_and_invalidate(); return Unavailable
   if dirty and !full_repair_of_committed(): return Error
   if !live_and_visible(): stop_and_invalidate(); return Unavailable
   result = surface.draw(target)     # synchronous; injection can detach
   if !live_and_visible():
      stop_and_invalidate(); dirty = true; return Unavailable
   if result is Error:
      stop_and_invalidate(); dirty = true; report_once()
      attempt_one_full_repair_now_if_attached(); return Error
   committed_model = target.model; committed_slice = target.index
   return Ok
```

Immediately **before and after every synchronous draw**, and before and after any full repair, check root `is_connected()` and panel visibility. These checks bracket the setter and overlay updates, including an injected detach between individual DOM writes. A detached element can still accept DOM writes in the current library; absence of an error is not proof of attachment. Fail closed means no logical cursor/model commit, stop Play, cancel/invalidate host work, mark visual status dirty/unavailable and issue a single host-unavailable announcement. Never intentionally initiate Scene operations on a detached root. If a synchronous fault hook detaches during a draw, fail after the draw without committing; the DOM may be partially written, and a future attached repair must restore the committed projection. To recover on reattach, verify connected, then redraw the last committed model in the same supported Scene; for replacement root, dispose old Scene and rebuild with saved view/focus before any next commit.

Each Play/Stop cycle advances `playback_epoch`. A history/configuration change advances `scenario_generation`. Every seek replacement advances `seek_token`. Host callbacks carry captured tokens, hold weak controller refs and return inert if stale, hidden, disconnected or disposed. At most one timer plus one seek rAF is live; schedule the next timer only after a successful render commit.

6.2 Gallery hide, show and page lifecycle

Signal	Required action	Resumption policy
Gallery panel hidden or another selected	Best-effort async pause/cancel plus synchronous visibility guard before effects; increment epoch and seek token.	One already queued callback may enter but cannot mutate. No automatic Play when shown.
Gallery panel visible again	Check host connected and redraw if dirty; restore control focus, otherwise retain view.	User resumes explicitly with Play; resume from same committed slice.
document hidden / pagehide	Perform the same stop and invalidation. On actual page unload, teardown and release listeners.	On page show, remain paused unless new instance starts.
Detached root / replaced host	Fail-closed guard before draw. Stop and retain last committed state.	Only repair/remount after host is connected.
Permanent teardown	Mark disposed first; cancel handles; detach listeners; release scene/controller owners.	Never resume the old instance; late callbacks inert.

The gallery currently builds panels once and hides them; hiding is not teardown. A pagehide listener alone cannot detect gallery-tab switching. S0 must test whether the gallery can provide a synchronous activation/deactivation callback; if not, use an IntersectionObserver or mutation observer as an asynchronous hint **plus a synchronous visibility predicate** at every timer/frame entry, before scheduling, before draw and before commit. The predicate checks connectedness, document visibility and actual ancestor panel-hidden state; a mere root attachment check is

insufficient. A callback already dispatched when hide occurs may execute once, but must perform **no simulation advancement, DOM writes, cursor commit, focus or notice changes**. Do not promise that no browser callback executes while hidden; promise that it is inert. Show never auto-resumes.

6.3 Failure and repair contract

Rendering: if any write fails, no new logical commit occurs. Invalidate the cached presentation, stop playback, report one error, then make exactly **one immediate, bounded full redraw** of the last committed model if the root is connected. On success clear dirty but leave playback stopped and requested action unsuccessful; on another failure remain dirty/stopped and expose Repair. Avoid retry loops or reentrant controller actions. A later Next/Previous/Restart, even at a boundary, attempts dirty repair before a no-op.

Scheduling: if scheduling a timer fails, keep the last committed display and stop; if requesting the seek frame fails, preserve the cursor and offer Retry. Do not hide an unbounded synchronous seek behind a fallback. A small measured fallback may be added later with an explicit safe work bound. **Setters:** a full redraw only heals a setter once its own repeat-write semantics have passed a fault sweep; before that, keep it disabled and use the audited application-owned overlay.

Use a manual scheduler with held cancelled callbacks and forced timer/frame failure, as the Petri demo already does. Include Pause→Play, hidden→visible, scenario switch and detach/remount races. Verify that the stale callback changes neither logical state, DOM, notices nor focus and retains no strong controller cycle.

7 Fault sweeps and written recovery contracts

Treat injected host/DOM faults separately from simulated ServerFail. The existing harness already has `perf/fault-contracts.json` with `nothing_committed`, `partly_committed` and `undocumented`. Extend that file and its validated fault-ID taxonomy rather than inventing an unrelated failure table. For overlay draws, logical cursor remains unchanged even when the DOM is partly committed; the new contract must state both facts explicitly.

Point / new fault ID	Initial expectation (to validate)	Retry and stop condition
service_overlay_write	partly_committed DOM only; logical target uncommitted.	One immediate full re-render of committed projection; if it fails, stopped/dirty.
service_control_aria	partly_committed visible/AX attributes; cursor unchanged.	Restore last committed content + ARIA; no automatic Play.
service_timer_request	nothing_committed when request fails; callback does not exist.	Stop with last committed state and announce once.
service_frame_request	nothing_committed target on failure.	Stay at cursor; explicit seek retry.
service_gallery_hidden	nothing_committed after deactivation.	Invalidate handles; show remains paused; user presses Play.
service_detached_before_draw	nothing_committed: app guard refuses detached root before writes.	On attach, redraw last committed projection; no cursor advance.
service_detached_during_draw	partly_committed: fault hook removes root mid-write; app logical state unchanged.	Post-draw guard refuses commit; attached full repair restores model.
service_teardown_late_tick	nothing_committed after disposal.	Old callback must be inert; zero live handles/listeners.
set_data_values / set_plot_items	undocumented until sweep; no assumed idempotence.	Data setter must be proven and enabled for final demo acceptance.
set_edge_label / set_text_input_value	undocumented until separate write sweeps.	Not a runtime projection dependency before the sweep.

Contract publication rule: these IDs are proposed test-plan names, not documented contracts today. Only add an entry marked `nothing_committed` or `partly_committed` to `perf/fault-contracts.json` after the implementing code and explanatory comments exist, with exact source file/line references that pass

`perf/lib/test-contracts.mjs`. Until then, label the point `undocumented` and do not count an observed recovery as a documented contract pass. A detached Scene remains unsupported; the before/after guards are explicitly application contracts.

Mandatory audit order: first sweep `set_data_values`, `set_plot_items`, `set_edge_label`, and `set_text_input_value` for every injectable DOM call, including first, middle, final write and repeated full repair. Assert SVG content, model/DOM equality, selection, pan/zoom, DOM-id stability, and no unintended duplicate. Only then use setters in a sustained dashboard. Reuse `perf/page/fault-injector.js` and existing page/recovery sweep logic. If an optional path cannot be audited, leave it `undocumented` and use the overlay. `set_data_values` is not optional for final sustained-library evidence: fix and resweep it or stop before S5 acceptance.

WASM exception accounting: use the harness's existing wasm-bindgen externref-slot tracking for DOM calls that fail while returning objects. Distinguish an expected lost-slot delta per injected failure from a surviving resource leak. An unexplained slot loss or any surviving listener/callback violates the endurance gate. Fault toggles are test-only; clean performance sampling must never include them.

8 Harness integration: clean W6, recovery through W5

8.1 Current baseline and explicit compatibility work

Baseline is **main 65511d1c64c3, crate 0.2.48, harness design revision 6**. Harness Phases 0–5, recovery/endurance runners, fault-contract checking, comparison, baselines and perf-smoke already exist. Do not reimplement those components or confuse simulator delivery stages with harness phases. In current `perf-harness/src/manifest/mod.rs`, W1–W5 are enumerated; `[recovery]` is accepted only for W5 and W5 requires graph edges. Those limitations are explicit change targets, not assumptions that W6 can already use them.

Chosen adapter: add `Workload::ServiceSimulation` (W6) for correctness, performance and endurance. For injected faults, keep `Workload::Recovery` (W5) but add a validated `recovery.target = "service_simulation"` discriminator and a simulator scenario/fixture reference. W5 selects simulator-specific fault IDs from the same `fault-contracts.json`, without changing old graph recovery manifests. The graph.edges requirement applies only to W5 graph target; no fake connector is demanded for simulator fault tests. Extend typed FaultPoint, JSON schema, validation, runner dispatch and expectation comparison atomically; no duplicate recovery mode or independent [service_recovery] convention.

W5 effort is a second executor, not just a discriminator. Its service path must construct and own a simulator controller/surface, inject DOM faults and retained cancelled callbacks, take pre/post structured app/DOM/AX checkpoints, compare logical history against the separately coded JavaScript oracle, classify contract outcomes and cleanly release all handlers. Reuse the harness injector, result writer, Chrome session, diagnostics, and externref-slot attribution, but not the graph twin-scene executor or graph-only reference model. Plan separate schema, typed validation, runner dispatch, failure sweep, Layer B/C integration and end-to-end CI tests. Preserve the existing W5 graph branch unchanged for legacy manifests (default target `graph`).

8.2 Manifest examples (proposals, not accepted today)

```
# W6: clean simulation; proposed TOML addition
schema_version = 1
workload = "service_simulation"
seed = 20261106
prng = "splitmix64"
[simulation]
scenario = "mixed_outage"
servers = 3
queue_capacity = 12
max_attempts = 2
retry_deferral_cap = 4
min_pops = 10000         # independently requires >=536 nonempty slices for A/B
batch_slices = 4         # only for Layer A/B observations
# no iteration is an individual heap pop
[execution]
modes = ["correctness", "performance"]
warmup_iterations = 10  # A/B four-slice batches, after 256 priming slices
measurement_iterations = 60  # A/B batch observations; C has separate real controls
repetitions = 3         # perf-smoke: diagnostic comparisons only
```

```
# W5: fault sweep targeted at the simulator; proposed
schema_version = 1
workload = "recovery"
[recovery]
target = "service_simulation"   # new typed discriminator
scenario = "E4"
# points use existing typed {point, expect} entries and
# perf/fault-contracts.json expectations: see section 7
# No graph.edges requirement for this target.
# The W5 target gets its own executor/oracle, not graph twins.
```

Maintain backwards compatibility with every existing W1–W5 manifest and result, plus schema rejection of mistyped new keys. If the W5 schema cannot encode the extension additively, change the schema version explicitly with migration tests; do not alter the meaning of a current version silently. Keep W6 operation-stream, actual event-pop and model-checkpoint hashes separate. Extend results with simulator input counts, actual pops and stale fractions.

8.3 Fixed W6 observation contract

An **iteration is one timed observation**, not a popped event, a single slice, or an entire replay. **Layer A:** four consecutive nonempty timestamp slices, timing only the pure Rust engine and its replay. **Layer B:** four consecutive synchronous `render_then_commit` calls, including `set_data_values`, overlay writes, cheap connectivity guards and synchronous commit bookkeeping between draws, but **excluding engine replay** and any harness-induced layout-reading visibility predicate. The input models and expected projections are prepared outside the B timer. Any measurement or layout work inside `set_data_values` is genuine library cost and remains inside the interval; label the result synchronous API cost, never a frame-time metric. **Layer C:** measure real accessible Next and Play controls as two separate series, one committed slice per observation, without a fixture-only batch-step control.

Smoke profile: follow the existing per-commit `perf-smoke` convention of **three independent fresh-browser repetitions**. For A and B, use 10 untimed warm-up observations and **60 measured four-slice observations per repetition**. With 3 repetitions, the comparison is explicitly diagnostic/inconclusive under the current five-repetition rule; do not report a statistical pass. An optional, separately named five-or-more-repetition comparison profile may support statistical verdicts, never be silently added to per-commit smoke. Sixty observations allow a median but not p95 or p99 under the current sample minima; preserve raw values and batch definitions.

Fixed trace windows: precompute the complete deterministic trace; slices 1–256 are untimed priming, not proof of steady state. A/B then consume slices 257–296 as warm-up and 297–536 as measurement: **at least 536 distinct nonempty slices** required. Layer C independently resets to the cursor after slice 256 for each control series, then takes 10 warm-up and 30 measured real commits (through slice 296): **at least 296 nonempty slices per series**.

Reject insufficient traces before timing; `min_pops=10000` is an independent minimum and never substitutes for the slice count. Do not wrap, reuse or silently shift a window.

Activity and steady-state preflight: compute the independent-reference metrics on the fixed **240-slice A/B measured window (297–536)**, before obtaining timing results. Occupancy means end-of-slice waiting count / queue capacity. Provisional bounds: mean occupancy [0.25, 0.90]; nonempty waiting queue in at least 50% of slices; mean actual pops/slice including stale [1.5, 32]; at least one UpBusy server in at least 50%. For chronological halves of 120 slices, require absolute difference of mean occupancy ≤0.20 and difference in mean pops/slice ≤25% of whole-window mean. These are activity and half-window stability tests, **not** proof of stationarity. The shorter C window (267–296) records the same activity counters but does not apply the 120-slice-half criterion.

S0 seed-qualification gate: before W6 timings and before fixing its default manifest, run the generator on exactly **500 predetermined seeds 0–499** with a frozen parameter set; publish per-seed refusal reasons and aggregate occupancy, busy and pop distributions. At least **475/500 (95%)** must qualify under the preceding bounds and satisfy both the 536-slice and 10,000-pop minima. If not, revise the generator parameters and rerun the entire same seed panel before any timing; never tune thresholds to retain selected seeds or select a favourable subsequence. Preserve all candidate qualification results in S0 evidence, including failures. These figures are **target gates not measured claims**: the existing S1 small-case generator has not been evaluated against them.

Reporting: every W6 result records the generator revision, seed, qualification status, both half-window metrics, occupancy histogram/mean, nonempty and UpBusy fractions, pop density and minimum-count checks, even when refused. A refusal is not a performance pass and produces no time series.

Layer C control attribution: Next measures real accessible Next input delivery → harness presentation boundary. Play measures **timer callback entry → corresponding committed slice presentation boundary** (scheduled-commit-to-presentation), not Play click → first tick. At S0 choose a **harness-owned wrapping Scheduler**: it stamps entry immediately before invoking the product timer callback, outside product code. Do not add a release-build instrumentation hook. The configured playback interval and click-to-timer wait are recorded separately, never included in Play latency. The frame attribution boundary and reference clock must match existing Layer C practice.

One-tick Play/Pause protocol: force the slowest **1 slice/second** product speed. For each observation use the real Play control, wait for exactly one successful timer-driven slice and its presentation boundary, then press the real Pause control outside the timed interval. Check committed cursor = old cursor + 1; epoch advanced on Pause; after the cancellation-settle boundary there is no further commit, callback work or pending timer. A second tick before Pause is **invalid_protocol_extra_tick**: reject the entire repetition, retain its trace, reset from slice 256 and investigate; do not silently count two, omit one, retry an individual observation or alter 296-slice accounting. Repeat 10 untimed and 30 measured one-tick cycles, ending at slice 296. Failed, hidden and stale ticks never enter a latency series.

Play runtime budget: 40 cycles at 1 Hz entail at least **40 seconds of timer delay per repetition**, so three repetitions require at least 120 seconds before CDP, presentation, teardown and start-up overhead. Require a per-repetition `wall_ms` of at least **90,000** and allocate at least **six minutes** if running all three in one CI job, with the real measured CI duration recorded at S0. Until the CI budget is verified, **schedule Play on nightly/manual W6 Layer C** and keep three-repeat A/B/Next diagnostic smoke per commit; missing Play coverage is reported as not run, never pass. Do not shorten the 1 Hz interval to hide protocol races.

Timing guard boundaries: at callback entry, check connectedness and evaluate the actual ancestor-hidden predicate (which S0 must select and characterise) outside any A/B timed interval. Within a B timed draw, rely only on a cached activation flag plus cheap `isConnected` checks before and after the synchronous write; never call `checkVisibility()`, `getComputedStyle`, `getBoundingClientRect` or equivalent layout-reading APIs inside B. If visibility changes during a synchronous fault hook, the post-draw guard must fail closed. In normal browser execution the synchronous callback-entry check catches a panel hidden before the observer delivered its notification.

For every A/B batch, record actual popped-event count (including stale), four slice indices, setter call count and end-of-batch checkpoint outside the timed interval. B starts before the first Scene/overlay write and ends once the fourth render and synchronous commit bookkeeping return; the clock includes cheap isConnected/cached-flag guards between writes and any internal set_data_values work, but excludes model preparation and the visibility/layout probe. Stale-only slices still change the mandatory data node’s slice ordinal, so the browser

attribution trace must observe a meaningful content mutation; test this explicitly. Clean W6 performance contains no injected faults; checkpoint mismatch invalidates timing. S0 must test the 3-repetition diagnostic classification and both distinct C control series before implementing the runner.

8.4 Layer and mode boundaries

Programme	A: pure Rust	B: WASM/synchronous DOM	C: real browser
W6 correctness	Rust/JS exact slice comparison; E0 tuple.	Fixed-slot content, delta=full and accessible state.	Real Next/Seek/Pause and view/focus invariance.
W6 performance	Four-slice batches, pure engine replay.	Four draws plus commits; internal setter work included; no replay/layout probe.	Next: real input→presentation. Play: wrapper at timer entry→presentation, 1 Hz; nightly until budget verified.
W6 endurance	Bounded heap and state stream.	Repeated use/destroy, deterministic element/handler return.	Resource counters, heap trend + explained externref losses.
W5 recovery: simulator target	Separate JS oracle execution; no graph twin scene.	Service-specific fault executor + full app/DOM checkpoint.	Hide, detach/remount, mid-step teardown, late tick.

8.5 Workloads, metrics and gates

Workload	Count contract	Projection obligation
E0	Exactly 19 pops, 12 slices, 13 snapshots, 5 stale.	Every snapshot must match independent JS and stable projection.
1k-pop trace	At least 1,000 actual pops; not enough alone to qualify for smoke.	Engine/oracle only at S1; no UI or setter claims.
10k-pop trace	At least 10,000 actual pops; W6 smoke also needs 536 slices.	Data setter on every displayed slice, not merely 1,000; measure sparse checkpoints separately.
100k-pop trace	At least 100,000 actual pops.	Pure streaming/oracle at scale; sampled DOM is not a 100k-crate-update claim.
E4 / W5	Injected faults, not mixed with clean timing.	Failure contracts and exact recovery measured separately.

Store actual arrivals, processed pops including stale, timestamp slices, sampled projection writes, heap peak, total live jobs, listener/callback counts, DOM elements by type, checkpoint bytes and externref lost slots with cause. A fixture named “10k events” must have at least 10k actual heap pops (not arrivals). Preserve current harness rule: any wrong checkpoint makes the run `invalid_incorrect` and invalidates its timings. Statistical comparison remains reporting except where the existing, explicitly configured baseline gates apply; raw samples and environment keys follow the existing rev 6 harness.

Use current existing perf-smoke, comparison and baseline promotion workflows. The simulator stage does not promise performance improvement without measured data. The initial correctness test is an additional per-commit blocking gate; browser resource/lifecycle acceptance requires a real CI runner, not a local script alone.

9 UI, controls and accessibility

Keep the fixed outer SVG/control shell when switching histories. A visible clock/timestamp readout is always identified as **simulated time**; playback speed uses **timestamp slices per wall-clock second** (1, 2, 5, 10) and does not alter the simulation rules. Next/Previous/Restart, range Seek, Play/Pause, scenario/config selectors, queue priority legend, summary counters , a “Repair display” action and a clear paused-while-hidden status are labelled by accessible names, not only icons.

Selection and focus are separate: selecting a job/server changes its detail pane but not the simulation; pointer focus never inadvertently advances time. Use roving keyboard focus across visible service/queue nodes, a selected logical

job ID that survives reordering, and a fallback focus target when the selected job leaves the visible window. Announce manual commits and errors politely; while playing, throttle routine status announcements to a documented cadence and always announce pause, completion or a fault. Disabled controls include a reason at boundaries.

Every displayed numeral, progress bar and SVG relationship comes from the same pure projection; assert label/value/accessibility-tree agreement. Do not use colour alone to express Ready/Running/Down/RetryWait. Scenario changes preserve focus on the equivalent control in the fixed shell, not on a removed scene node.

10 Test programme and acceptance

Family	Minimum test / deterministic criterion
Engine + E0–E3	E0 exact 19/12/13 and five stale; E1 running cancel/timeout; E1-RC 6/6/7; E2-RT 5/5/6; E2 both Failed reasons including fifth deferral; E3 duplicate-outage and mixed-source ordering.
Independent oracle	Separately coded JavaScript sorted-array model, no shared Rust core. >=500 seeds, 2–4 servers, bounded queues; compare every slice, event-pop keys, terminal states, counters and canonical hashes. Run Rust native and wasm32.
Projection + mutation audit	Pin .svg-dom-graph-content with library test; sweep set_data_values before enabling mandatory live data node; delta/full and 1,000 sustained setter commits.
Host and gallery	Detach before and via injected mid-draw hook; pre/post is_connected gates; asynchronous hide callback may run once but is inert; show remains paused.
Controller/cancellation	Keep-cancelled timer and frame callbacks, pause/resume, seek replacement, one-slice forced cooperative replay, speed change and scenario switch: no stale commit/notice/focus.
Fault recovery	One immediate bounded repair after render error; if it fails stay stopped/dirty. Later boundary no-op triggers repair. W5 contract output and externref accounting match.
Endurance	Repeated construct/use/hide/show/destroy; DOM and listeners return to baseline after teardown, outstanding callbacks zero; heap slope and wasm lost slots explained.
Harness & CI	W6 A/B = 60 four-slice observations; C = 30+ Next and 30 Play/Pause (1 Hz); Play nightly pending budget; 500-seed ≥95% S0 qualification; W5 second executor.

RELEASE GATE

Do not claim simulator production readiness from a local smoke test. Completion requires native and wasm32 oracle matches, **1,000 successful sustained Scene::set_data_values commits**, the setter fault sweep, attached-host/gallery cases, W5 service-executor recovery-contract gates and W6 correctness on real CI. The existing harness Phase 5 is already complete and is not repeated here.

11 Simulator delivery stages (S0–S5)

Stage	Deliverable	Exit / stop-go decision
S0 — approved audit	Setter/class hook and gallery predicate; W5 executor; W6 generator qualification; 1 Hz Play/Pause with wrapping scheduler and explicit CI cost.	500 seeds, ≥95% qualified before timing; mark outside product; extra-tick invalidates repetition; record budgets.
S1 — pure engine	Unpublished service-sim-engine crate; Rust BinaryHeap; separately coded JS sorted-array oracle; E0, E1-RC, E2-RT and E1–E3; discrepancy ledger.	Native, 32-bit WASI and CI/clippy tests; independently adjudicated >=500 seeded runs; freeze only agreed hashes.
S2 — projection	Long-lived fixed Scene, overlay, reserved data node; exact incremental projection; class compatibility test.	E0 projection exact; stable focus/view, no DOM growth; when setter enabled call it on every displayed slice.
S3 — controller	Manual clock, epochs/tokens, synchronous hidden predicate, pre/post-draw host checks, forced mid-draw detach.	One queued post-hide callback inert; full/step/seek equivalent; no post-detach commit.
S4 — recovery	Audit/repair required data setter, new service-specific W5 executor, JS oracle checkpoints, source-cited contracts.	No undocumented pass; all injected write points recover or stop as documented; externref accounting correct.
S5 — harness evidence	W6 A/B four-slice observations; C real Next/Play; 3-repeat diagnostic smoke and optional 5+ comparison; 1k/10k/100k pops; mandatory setter.	CI/schema correctness pass; setter updates on every displayed S2 slice (1,000 minimum); statistical verdicts separated.

12 S0 questions and constraints

Required S0 decisions, in order: (1) sweep mandatory `add_data_node + set_data_values` and establish fixed-slot compatibility; (2) pin the DOM hook and any semver/changelog change; (3) choose a synchronous gallery-hidden ancestor predicate and activation hint; (4) preflight 500 fixed seeds with ≥95% W6 qualification on the 240-slice A/B window, and freeze generator/manifest only afterwards; (5) validate three-repeat A/B/Next smoke, its Layer B boundaries, plus harness-owned timer mark, 1 Hz one-tick Play/Pause, extra-tick rejection and separate Layer C runtime budget (nightly until affordable); (6) design the W5 simulator executor and source-cited contracts; (7) measure replay before checkpoint/chunking machinery. S0 gates later UI/harness stages, not the pure S1 engine and oracle.

Important separation: simulated ServerFail is business logic. DOM write failure is injected infrastructure. A rendered SVG state is not a model commit until the full Surface operation succeeds. Detached elements can accept writes, so always check connectedness. A full redraw repairs only into an existing attached Scene and must preserve view and focus. A mid-draw injected detach is detected by the post-draw guard and cannot advance the cursor. Scheduled work stops on gallery hide even if panel is never destroyed.

Stage status: S1 development can continue while S0 measurement preflight is specified. The current downloadable package is a source-only, not-yet-repository-integrated starter with Rust heap engine, separately coded JS sorted-array reference, known-answer fixtures and a differential runner. The same author coded both language versions; treat them as separately coded, not fully independently validated. A third reader has checked E0/E1-RC/E2-RT fixture counts from the text, but native compilation, wasm32-wasip1 cross-check, ≥500 Rust/JS seed agreement, canonical binary digest freeze and CI integration remain unverified. W5/W6 runner changes are not prerequisites for S1.

Do not drift: “event” always means a popped heap event, including stale; input/event/checkpoint hashes stay distinct; no timing becomes a deterministic gate; no public library API before demonstrated necessity. Preserve existing `changelog.md` entries and normally approved release bookkeeping only.

Appendix A Canonical state and hash contract

For equality, encode structured state in a fixed field order and sort all logical-keyed collections. Include the clock, batch ordinal, all live job states and attempt tokens, server states, waiting order, retry state and terminal failure reasons, future heap in sorted event-key order, event insertion sequence, external generator cursor and counters. Serialize unsigned integers little-endian; encode enum variants with explicit stable discriminants and strings with length prefixes. Compute SHA-256 of bytes and store both the hash and a structured mismatch report. DOM IDs are checked for uniqueness but not equated across complete scene rebuilds until proven stable.

```
checkpoint(version=1) = SHA256(
    config_id || seed || tick || batch_index || generator_cursor ||
    counters || sorted_jobs_with_failure_reason || sorted_servers || waiting_order ||
    sorted_future_events || next_insertion_sequence
)

separate hashes:
  input_stream_sha256  : generated external operations
  event_pop_sha256     : processed event key/payload + stale/active bit
  model_checkpoint_sha : canonical post-slice continuation state
  dom_projection_sha   : canonical rendered content, not innerHTML
```

For a renderer checksum, use logical element keys, text/attributes, geometry to the harness's documented tolerance, accessibility roles/names/relationships and relevant paint order. Ignore inconsequential DOM child order and unstable generated IDs. The engine checkpoint is exact; floating visual coordinates use the harness's existing tolerance policy.

Appendix B Source baseline and reproducibility

Repository verified 6 November 2026: `ChrisWhealy/svg-dom-graph` main `65511d1c64c3641b96d733da538f14230de8e1cd`, Cargo version `0.2.48`. `perf/README.md` is now a navigation index; design revision 6 and completed harness Phases 0–5 are documented there, with details in `perf/docs/phase-3-gallery.md`, `perf/docs/phase-4-recovery.md` and `perf/docs/phase-5-comparison-and-baselines.md`. Current source includes `perf/fault-contracts.json`, `perf/page/fault-injector.js` and `perf-harness/src/manifest/mod.rs` with W1–W5 and the graph-oriented W5 [recovery] restriction.

For projection precedent use `demo-app/src/petri_net/scene.rs`, its `controller/{mod,scheduler}.rs`, and `projection/mod.rs`. Scene fixed topology is established at creation. The overlay uses application-owned DOM nodes; this is a demo implementation pattern, not a public Scene visibility/mutation promise.

Historical inputs: `design-specs/performance_harness.pdf` currently rev 6, `design-specs/petri_net.pdf` and the 4 November 2026 functional-coverage audit. Reviewer E0 independent run confirmed 19 pops, 12 slices, 13 snapshots and 5 stale events. No simulator implementation, W5 simulator executor, W6 workload, mandatory setter sweep or refreshed fixture hashes are claimed to exist solely because of this document. E1-RC and E2-RT are newly specified regression probes, pending independent implementation.